Experiment 1: Hadoop Installation and File Management

Big Data Analytics (BIS701) — Laboratory Record

Objective

Install Hadoop and implement file management tasks including adding, retrieving, and deleting files/directories in HDFS.

Prerequisites

- Linux/Ubuntu OS (Recommended: Ubuntu 20.04 or later)
- Java JDK 8 or 11
- Minimum 4 GB RAM

Step 1: Install Java

```
# Update system packages
sudo apt update

# Install OpenJDK
sudo apt install openjdk-8-jdk -y

# Verify installation
java -version
javac -version

# Set JAVA_HOME
echo "export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64">> ~/.bashrc
echo "export PATH=$PATH:$JAVA_HOME/bin">> ~/.bashrc
source ~/.bashrc
```

Step 2: Create Hadoop User

```
sudo adduser hadoop
sudo usermod -aG sudo hadoop
su - hadoop
```

Step 3: Install SSH and Configure Passwordless SSH

```
sudo apt install openssh-server openssh-client -y
ssh-keygen -t rsa -P ""
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 640 ~/.ssh/authorized_keys
ssh localhost
```

Step 4: Download and Install Hadoop

```
# Download Hadoop 3.3.6
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

# Extract
tar -xzf hadoop-3.3.6.tar.gz
```

```
sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R hadoop:hadoop /usr/local/hadoop
```

Step 5: Configure Environment Variables

Edit ~/.bashrc and add the following lines:

```
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"

# Reload
source ~/.bashrc
```

Step 6: Configure Hadoop Files

hadoop-env.sh

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
# Add:
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

core-site.xml

```
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://localhost:9000</value>
</property>
<property>
<name>hadoop.tmp.dir</name>
<value>/home/hadoop/hadoopdata/tmp</value>
</property>
</configuration>
```

hdfs-site.xml

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.namenode.name.dir</name>
<value>file:///home/hadoop/hadoopdata/namenode</value>
</property>
<property>
<name>dfs.datanode.data.dir</name>
<value>file:///home/hadoop/hadoopdata/datanode</value>
</property>
</configuration>
```

mapred-site.xml

```
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>
```

yarn-site.xml

```
<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>
```

Step 7: Create Required Directories

```
mkdir -p ~/hadoopdata/tmp
mkdir -p ~/hadoopdata/namenode
mkdir -p ~/hadoopdata/datanode
```

Step 8: Format NameNode and Start Hadoop

```
# Format NameNode (only the first time)
hdfs namenode -format

# Start Hadoop services
start-dfs.sh
start-yarn.sh

# Verify services
jps
```

Step 9: File Management Operations

Creating Directories

```
# Create directories in HDFS
hdfs dfs -mkdir /user
hdfs dfs -mkdir /user/hadoop
hdfs dfs -mkdir /input

# List directories
hdfs dfs -ls /
hdfs dfs -ls /user
```

Adding Files

```
# Create sample files
echo "Hello Hadoop! This is a test file."> sample.txt
echo "Line 1: Big Data Processing"> data.txt
echo "Line 2: Hadoop File System">> data.txt

# Copy file from local to HDFS
hdfs dfs -put sample.txt /user/hadoop/
```

```
hdfs dfs -put data.txt /input/

# Copy an entire directory
mkdir local_data
echo "File 1 content"> local_data/file1.txt
echo "File 2 content"> local_data/file2.txt
hdfs dfs -put local_data /user/hadoop/
```

Retrieving Files

```
# View file content
hdfs dfs -cat /user/hadoop/sample.txt

# Copy file from HDFS to local
hdfs dfs -get /user/hadoop/sample.txt downloaded_sample.txt

# Copy to current directory
hdfs dfs -get /input/data.txt .

# View file using copyToLocal
hdfs dfs -copyToLocal /user/hadoop/data.txt local_copy.txt
```

Deleting Files and Directories

```
# Delete a file
hdfs dfs -rm /user/hadoop/sample.txt

# Delete directory (non-empty)
hdfs dfs -rm -r /user/hadoop/local_data

# Delete directory (must be empty)
hdfs dfs -rmdir /test_dir

# Move to trash (recoverable)
hdfs dfs -rm /input/data.txt

# Permanent deletion
hdfs dfs -rm -skipTrash /input/data.txt
```

Additional Useful Commands

```
# Check file status
hdfs dfs -stat /user/hadoop/sample.txt

# Change replication factor
hdfs dfs -setrep -w 3 /user/hadoop/sample.txt

# Check disk usage
hdfs dfs -du /user/hadoop

# Check filesystem/cluster health report
hdfs dfsadmin -report
```

Result

Hadoop was successfully installed in pseudo-distributed mode, and basic HDFS file management operations — creating directories, adding, retrieving, and deleting files — were carried out successfully.

Steps 1–3: Getting the machine ready

- Java: Hadoop itself is written in Java, so it needs a JVM to run on — this is like installing the engine before you can drive the car.
- Hadoop user: a dedicated OS account, mainly so Hadoop's files/permissions don't get tangled up with your personal account.
- Passwordless SSH: Hadoop's master node talks to its worker nodes by silently logging into them over SSH to start/stop processes. Even in pseudo-distributed mode (everything on one machine), it SSHs into "localhost" as if it were a separate machine, so it needs a key that doesn't ask for a password — otherwise every startup would pause waiting for you to type one in.

Steps 4–7: Configuring Hadoop
This is the part examiners love asking about — each XML file configures one of Hadoop's core pieces:

- **core-site.xml** → says *where HDFS lives* (`fs.defaultFS`, the address of the NameNode) and where to keep temp data.
- **hdfs-site.xml** → HDFS-specific settings: `dfs.replication=1` means "don't bother copying blocks 3x since there's only one node," plus where the NameNode and DataNode store their actual data on disk.
- **mapred-site.xml** → tells MapReduce jobs to run using **YARN** as the resource manager (instead of the old standalone mode).
- **yarn-site.xml** → tells YARN's NodeManager how to run MapReduce's shuffle phase (`mapreduce_shuffle`).

Analogy: core-site.xml is the building's *address*, hdfs-site.xml is the *floor plan* (how storage is laid out), and mapred/yarn-site.xml are the *staffing rules* for who processes the work.

Step 8: Format + start

- `hdfs namenode -format` — like formatting a new hard drive. **Only do this once** (formatting again wipes existing HDFS metadata — a common viva trap question: "what happens if you format twice?").
- `start-dfs.sh` boots the storage layer (NameNode, DataNode); `start-yarn.sh` boots the processing layer (ResourceManager, NodeManager).
- `jps` just lists running Java processes — you use it to confirm all 5 daemons (NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager) actually started.

Step 9: File operations
This is just Linux-style commands prefixed with `hdfs dfs`, because HDFS isn't your normal filesystem — it's distributed, so it needs its own client to talk to it:

- `-put` / `-get` → copy local ↔ HDFS (like `cp` but crossing the boundary into the cluster)
- `-cat` → print file content
- `-rm` → delete (goes to trash by default, like a recycle bin; `-skipTrash` deletes for real)
- `-setrep` → change how many copies of a file's blocks are kept (fault tolerance)
- `-du` → disk usage
- `dfsadmin -report` → health/status report of the whole cluster